

MACHINE HAWK

Edge AI Predictive Maintenance Platform

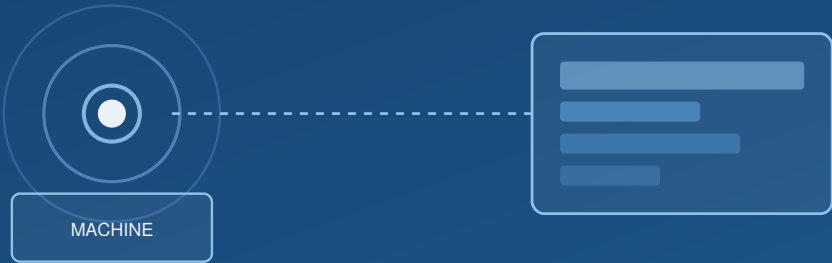
EDGE AI

ESP32

MQTT

REACT DASHBOARD

INDUSTRIAL IoT

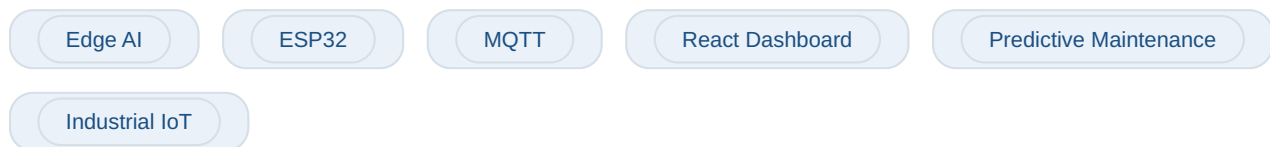


Document Type	Category	Status	Version
Engineering Prototype Documentation	Industrial IoT / Predictive Maintenance	Prototype — Hackathon Submission	Rev. 1.0

1. Project Overview

Machine Hawk is an Edge AI Predictive Maintenance platform built for industrial equipment. The system continuously monitors machine vibration and temperature at the point of measurement, runs an on-device TinyML model to score abnormal behaviour in real time, and streams the resulting telemetry to a backend service and web dashboard so that maintenance teams can act before a failure occurs.

The core objective of the platform is early anomaly detection: rather than waiting for a machine to fail or relying on fixed maintenance schedules, Machine Hawk evaluates sensor behaviour on the microcontroller itself and raises an alert the moment a reading departs from the learned normal operating pattern.



SYSTEM SUMMARY

A single ESP32 node samples temperature and vibration, runs an Edge Impulse TinyML classifier locally, and publishes structured telemetry over MQTT. A Python backend ingests this stream, computes a per-machine health score, and exposes it through a REST API consumed by a React dashboard used by the maintenance engineer.

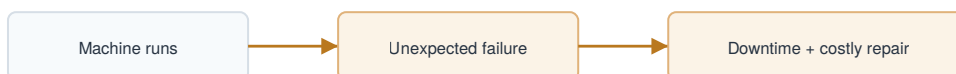
2. Problem Statement

Industrial equipment is typically maintained on a fixed schedule or repaired only after it has already failed. Both approaches carry avoidable cost and risk:

- **Unexpected machine failures** — faults are discovered only once the equipment has stopped, with no advance warning.
- **Production downtime** — unplanned stoppages halt dependent processes on the factory floor.
- **Reactive maintenance** — technicians respond after damage has already occurred rather than intervening early.
- **High repair costs** — failures caught late are more expensive to fix than issues caught in an early, developing stage.
- **Need for early anomaly detection** — teams lack a low-cost way to continuously watch equipment health at the sensor level.

Current vs. Proposed Approach

Reactive maintenance (current)



Predictive maintenance (Machine Hawk)

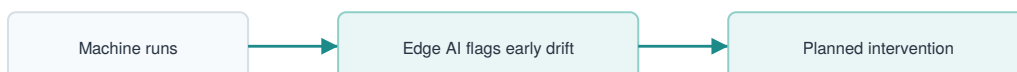


Fig. 2.1 — Reactive maintenance versus the early-warning path enabled by Machine Hawk.

3. System Architecture

Machine Hawk follows a linear edge-to-cloud-to-dashboard pipeline. Sensing and inference happen entirely on the ESP32; only compact JSON telemetry leaves the device. The diagram below shows the full data path from the physical machine to the maintenance engineer.

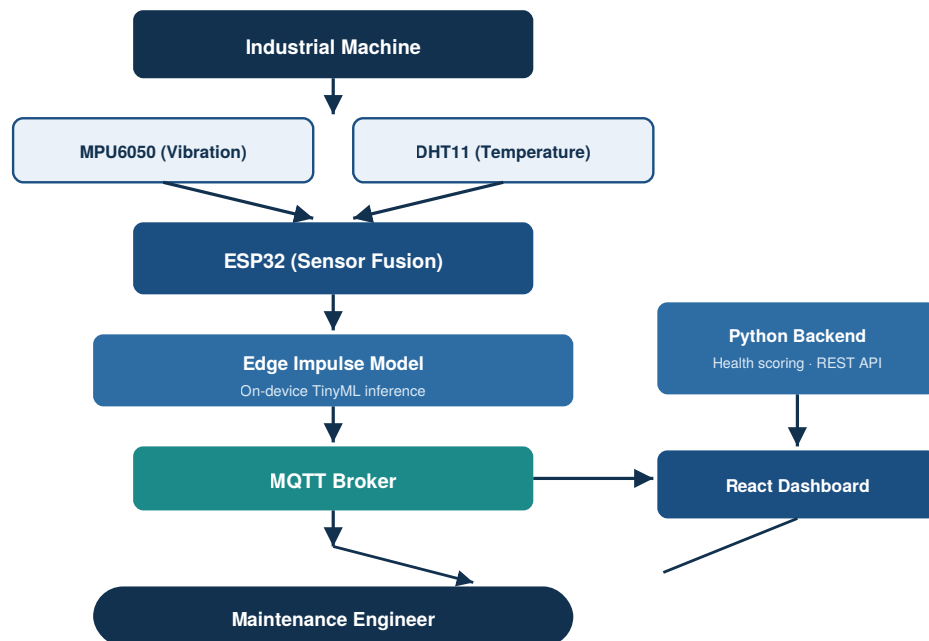


Fig. 3.1 — End-to-end Machine Hawk data pipeline, from sensor to decision-maker. The ESP32 both publishes telemetry over MQTT to the Python backend and reports independently to ThingsBoard for remote relay control.

Component Descriptions

Industrial Machine — the physical asset being monitored; vibration and heat are the two leading indicators of developing mechanical faults.

MPU6050 / DHT11 — an I2C accelerometer/gyroscope module and a digital temperature sensor mounted on the equipment to capture motion and thermal behaviour.

ESP32 — the Wi-Fi-enabled microcontroller that samples both sensors, maintains a short moving-average window, and drives the local safety relay and alert LED.

Edge Impulse Model — a TinyML anomaly classifier exported from Edge Impulse and run directly on the ESP32, so inference never leaves the device.

MQTT Broker — a lightweight publish/subscribe message broker that carries telemetry from the ESP32 to any connected subscriber, including the backend and ThingsBoard.

Python Backend — a FastAPI service that subscribes to the MQTT topic, maintains rolling per-machine telemetry, computes a composite health score, and serves it over a REST API.

React Dashboard — a TypeScript/React single-page application that visualizes machine health, trends, alerts, and recommendations.

Maintenance Engineer — the end user who monitors the dashboard and acts on the alerts and recommendations it surfaces.

4. Hardware Components

Component	Purpose
ESP32	Main controller; samples sensors, runs on-device Edge AI inference, and drives the alert LED and safety relay.
MPU6050	6-axis accelerometer/gyroscope connected over I2C; captures vibration data used for anomaly detection.
DHT11	Digital temperature sensor; feeds the moving-average trend forecast and threshold checks.
Relay	Optional emergency control output; can isolate the monitored equipment when a critical anomaly is confirmed.
Status LED	On-board visual indicator that mirrors the current alarm state of the machine.

NOTE

The current prototype uses a DHT11 temperature sensor and an MPU6050 for vibration sensing on a single ESP32 node. The architecture supports swapping in higher-precision sensors (e.g. a DHT22) without changes to the firmware's control flow.

5. Software Stack

Technology	Purpose
ESP32 Arduino Framework	Firmware layer for sensor sampling, actuator control, and Wi-Fi/MQTT networking.
Edge Impulse	TinyML model training and on-device inference SDK for anomaly classification.
MQTT	Lightweight publish/subscribe protocol carrying telemetry between the ESP32, backend, and ThingsBoard.
Python	Backend service (FastAPI) for telemetry ingestion, health scoring, and the REST API.
React	Front-end framework powering the maintenance dashboard.
Tailwind CSS	Utility-first styling for the dashboard's layout and components.
TypeScript	Static typing across the dashboard codebase for safer refactors and clearer data contracts.

6. Edge AI Workflow

The anomaly detection model deployed on the ESP32 is produced through the standard Edge Impulse development cycle, taken from raw sensor capture through to on-device inference:

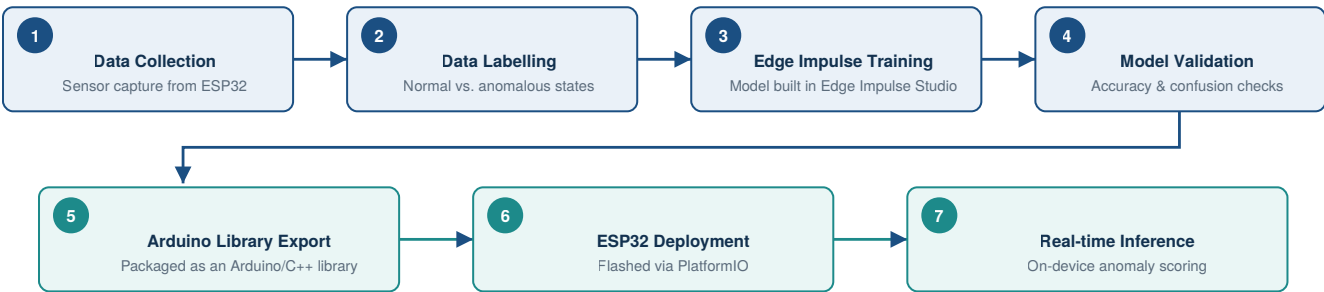


Fig. 6.1 — Model lifecycle from raw sensor data to on-device inference.

PLACEHOLDER

Model accuracy, confusion matrix, and validation figures are intentionally omitted here. Insert final Edge Impulse validation metrics once training on the deployed dataset is complete.

7. Operational Workflow

Once deployed, Machine Hawk runs the following cycle continuously on a fixed telemetry interval, turning a raw sensor reading into a maintenance recommendation on the dashboard.

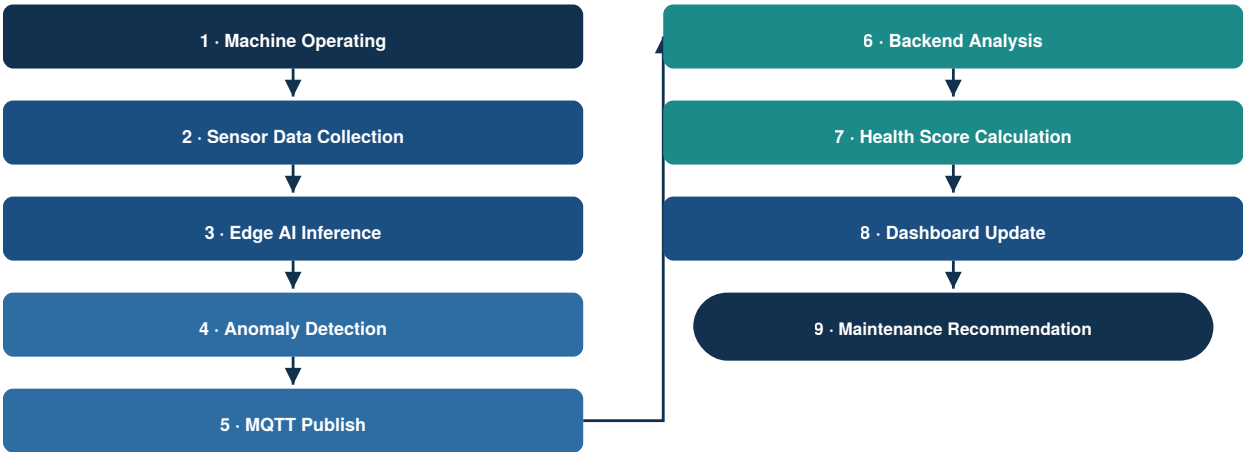


Fig. 7.1 — Closed-loop operational cycle, from live machine data to an actionable recommendation.

8. Dashboard Features

The React dashboard is the primary interface for the maintenance engineer. It presents live and historical machine data across the following views:

Overall Health Score

A single composite score per machine, derived from temperature, forecast temperature, and anomaly score.

Machine Status

Online / warning / offline state for every machine in the factory catalogue at a glance.

Failure Risk

Anomaly score and predicted temperature trend used to flag machines drifting toward failure.

Active Alerts

A live feed of warning and severe-anomaly alerts raised across the factory floor.

AI Recommendation

Generated maintenance guidance based on current machine health and recent trend.

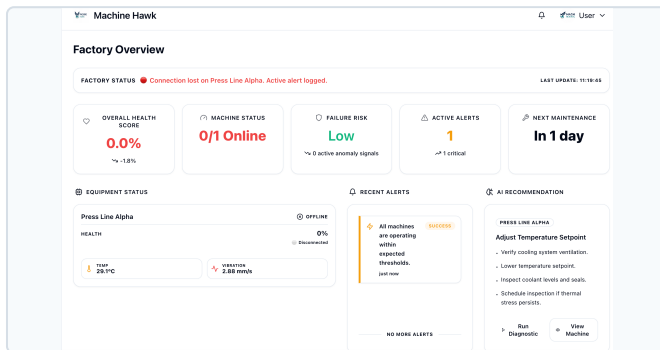
Maintenance History

A log of past preventive, predictive, and corrective maintenance actions per machine.

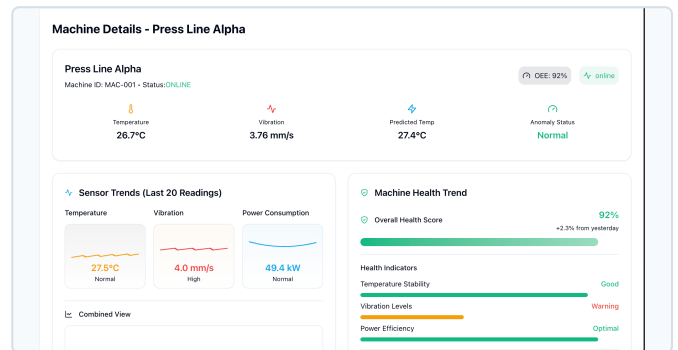
Reports

Production, efficiency, quality, maintenance, and energy reports available for review and export.

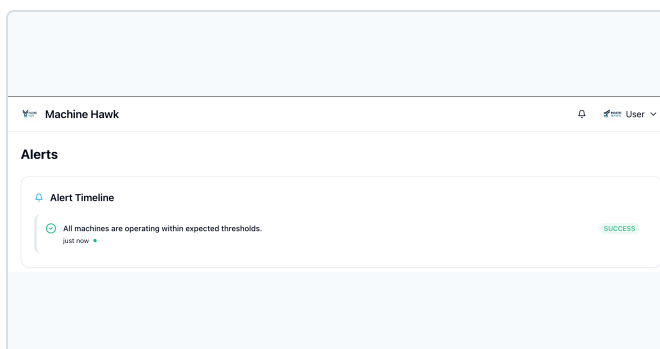
Interface Preview



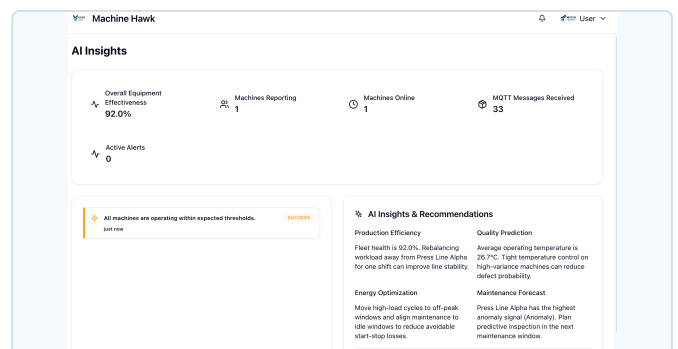
Factory Overview



Machine Details



Alerts



AI Insights & Recommendations

9. Key Features



Real-Time Monitoring

Continuous sensor sampling on a fixed telemetry interval.



Edge AI

TinyML inference runs directly on the ESP32.



Low Latency

Local inference avoids a round trip to the cloud for detection.



Predictive Maintenance

Trend forecasting flags problems before thresholds are breached.



MQTT Communication

Lightweight publish/subscribe messaging between device and backend.



Industrial Dashboard

Factory-wide visibility across every connected machine.



Scalable Architecture

The machine catalogue and MQTT topic scheme extend to additional nodes.

10. Future Scope

- **Multi-machine monitoring** — extend the current single-node prototype to a full factory floor of ESP32 nodes reporting to the same backend.
- **Cloud deployment** — host the backend and MQTT broker on managed cloud infrastructure for wider accessibility and durability.
- **Remaining Useful Life prediction** — move beyond binary anomaly flags toward an estimated time-to-failure for each machine.
- **OTA firmware updates** — push firmware and model updates to deployed ESP32 nodes remotely.
- **Mobile application** — a companion mobile app for on-the-floor alerts and quick status checks.
- **Industrial protocol support** — add Modbus and CAN connectors for integration with existing plant equipment.

11. Conclusion

Machine Hawk brings together embedded systems, Edge AI, industrial IoT connectivity, predictive maintenance logic, and a modern web dashboard into a single, end-to-end prototype. By running anomaly detection directly on the ESP32 and surfacing the result through a real-time dashboard, the platform gives maintenance teams an early, actionable warning — reducing unplanned downtime and shifting equipment upkeep from reactive to predictive.

SUMMARY

Embedded Systems · Edge AI · Industrial IoT · Predictive Maintenance · Modern Web Technologies